

CSP502 SL — Interim Winter 2026

Tutorial Exercise: Algorithmic Complexity

Algorithmic Complexity

You are already familiar with terms such as $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, and $O(n^2)$.

The purpose of this tutorial is not to repeat their definitions. Instead, you will examine:

- what Big-O actually tells us;
- what it does not tell us;
- how theoretical growth differs from actual execution time; and
- what evidence would be required before making a performance claim.

Part A — Big-O: What Are We Actually Saying?

No AI tools are to be used for Part A to E.

Consider the following four hypothetical running-time functions:

$$T_1(n) = 1000 n \tag{1}$$

$$T_2(n) = 5 n^2 \tag{2}$$

$$T_3(n) = 50 n \log_2 n \tag{3}$$

$$T_4(n) = n^2 + 500 n + 10000 \tag{4}$$

Without writing or executing a program, examine these expressions.

Questions

1. State the Big-O growth class that you would associate with each expression.
2. Which expressions have the same asymptotic growth class?
3. Arrange them according to their expected growth for sufficiently large n .
4. Does this ordering necessarily tell you which expression has the smallest value for:
 - $n = 10$?

- $n = 100$?
 - $n = 10,000$?
5. Can an expression belonging to $O(n^2)$ have a smaller value than one belonging to $O(n \log n)$ for some values of n ?
 6. If yes, investigate approximately where the relative ordering changes.
 7. What information present in the four expressions is *not* visible when we describe them only using Big-O?

Record one conclusion: From this exercise, what does Big-O preserve and what does it abstract away?

Part B — Reason About Code, Not Algorithm Names

For each of the following code fragments:

1. identify a suitable measure of input size;
2. identify the operation whose growth you are considering;
3. determine how the number of such operations grows with input size;
4. state the corresponding asymptotic behaviour;
5. record any assumption required for your conclusion.

Fragment 1

```
for i = 1 to n
    process(A[i])
```

Fragment 2

```
for i = 1 to n
    for j = 1 to n
        process(A[i], A[j])
```

Fragment 3

```
i = 1
while i < n
    process(A[i])
    i = 2 * i
```

Fragment 4

```
for i = 1 to n
    if A[i] == target
        return i
return NOT_FOUND
```

Questions for Fragment 4 Would you describe its behaviour differently for:

- the target at the beginning;
- the target near the end;
- the target absent?

What does this tell you about describing an algorithm using only one complexity expression?

Fragment 5

```
for i = 1 to n
    process(A[i])
for j = 1 to n
    for k = 1 to n
        compare(A[j], A[k])
```

Which part determines the asymptotic growth? Does that mean the other part has no practical cost?

Analyse the distinction.

Part C — Examine the Claims

For each statement below, classify it as:

Justified / Unjustified / Conditionally Justified

Your classification alone is not sufficient. Give the reasoning and, where appropriate, state what additional information would be required.

Claim 1 Algorithm A is $O(n)$, whereas Algorithm B is $O(n^2)$. Therefore, Algorithm A will always execute faster.

Claim 2 Two algorithms are both $O(n \log n)$. Their execution times should therefore be approximately equal.

Claim 3 When the input size was doubled, execution time approximately quadrupled. Therefore, the algorithm is $O(n^2)$.

Claim 4 An $O(n^2)$ algorithm should never be selected when an $O(n \log n)$ algorithm exists for the same problem.

Claim 5 Program A completed in 0.8 seconds and Program B in 1.2 seconds. Therefore, Algorithm A is more efficient.

Claim 6 Big-O tells us how an algorithm's resource requirement grows with input size; it does not tell us directly how many seconds a particular implementation will take.

After considering all six claims, identify:

Which claim would be most dangerous to accept without qualification while making a real software-design decision? Why?

Part D — Same Big-O, Same Performance?

Consider two hypothetical implementations whose running times approximately follow:

$$T_A(n) = 20n \quad \text{and} \quad T_B(n) = 200n$$

Questions

1. What is the Big-O complexity of A and B?
2. Does Big-O distinguish their actual amount of work?
3. If both algorithms process one million items, could the difference matter in a real system?
4. Does this make Big-O incorrect or incomplete for this particular question?

Now consider:

$$T_C(n) = 2n^2 \quad \text{and} \quad T_D(n) = 100n \log_2 n$$

Without immediately calculating every possible value:

5. Which has the better asymptotic growth?
6. Must it therefore be faster for every n ?
7. Estimate or investigate a region in which their relative performance changes.

What would you call such a region?

Part E — From Mathematics to a Computer

So far, the discussion has involved mathematical expressions and operation counts.

Actual programs execute on real machines. Consider the following factors:

- constant factors;
- lower-order terms;
- programming language;
- compiler and optimization;

- function-call overhead;
- memory allocation;
- cache behaviour;
- characteristics of the input;
- operating-system activity;
- hardware.

Task

Classify these factors into two broad groups:

Factors primarily related to the algorithm/implementation
and

Factors primarily related to the execution environment

If you find that some factors cannot be cleanly placed into only one category, analyse why.

Then answer:

If Big-O abstracts away many of these factors, why is Big-O still useful?

Part F — Now Consult an AI Tool

Give the AI the following statement:

“Algorithm A is $O(n \log n)$, while Algorithm B is $O(n^2)$. Therefore, A should always be selected because it is more efficient.”

Ask the AI whether it agrees.

Do not ask it to rewrite your earlier answers.

Compare its reasoning with the reasoning your group recorded in Parts A–E.

Prepare a brief table:

Aspect	Your Group's Reasoning	AI's Reasoning	Agreement / Difference
Meaning of Big-O			
Actual execution time			
Small inputs			
Large inputs			
Role of constants			
Final claim			

Question

Did the AI introduce any consideration that your group had overlooked?

Did it make any statement that requires qualification or experimental evidence?

Part G — Design the Experiment Before Running It

Consider two implementations intended to solve the same computational problem.

Suppose theoretical analysis indicates:

Algorithm A: $O(n^2)$

Algorithm B: $O(n \log n)$

You now want to determine how they actually behave on your computer.

Do not execute anything yet.

Design an experiment that could test their behaviour fairly.

Your experimental plan should address:

- input sizes;
- characteristics of the input;
- how equivalent inputs will be provided to both algorithms;
- what will be measured;
- number of measurements;
- how measurements will be summarized;
- what should remain unchanged between experiments.

Before any execution, make three predictions:

Prediction 1: What do you expect for relatively small inputs?

Prediction 2: What do you expect as input size becomes large?

Prediction 3: Do you expect there could be a point or region at which the relative performance of the algorithms changes? Why?

Record these predictions. Do not modify them later to match experimental observations.

Final Reflection

Answer briefly:

1. Before this tutorial, what did the statement “Algorithm A has better Big-O complexity than Algorithm B” mean to you?
2. After completing the tutorial, what can you legitimately conclude from such a statement?
3. What can you *not* conclude without additional information or experimental evidence?
4. If theoretical analysis and experimental measurements appear to disagree, should one simply be rejected? What should you investigate first?

Submission

Each group should submit:

1. Responses to Parts A–E completed without AI assistance.
2. AI comparison from Part F.
3. Experimental design and predictions from Part G.
4. Final reflection.
5. Brief contribution statement for group members.